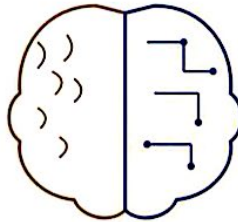
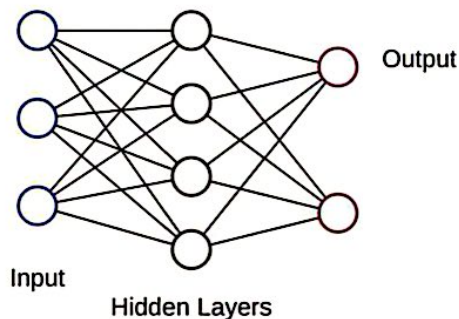


DEEP LEARNING

- CẨM NANG VIẾT TAY -

Nắm vững khái niệm. Xây dựng hệ thống thông minh.
Định hình tương lai cùng Deep Learning. ☺



“Learn Deep.
Think Big.
Build Smart.
Change the World.”

Feature Learning • Representation Learning • Automation • Scalability • Innovation

Nội dung bên trong?

- ① Giới thiệu về Deep Learning
 - ② Kiến thức cơ bản về Neural Networks
 - ③ Activation Functions
 - ④ Loss Functions
 - ⑤ Optimization Algorithms
 - ⑥ Convolutional Neural Networks (CNN)
 - ⑦ Recurrent Neural Networks (RNN)
 - ⑧ Long Short-Term Memory (LSTM)
 - ⑨ Transformers & Attention
 - ⑩ Generative Models (GAN, VAE)
 - ⑪ Deployment & Best Practices
- + Cheat Sheet & Interview Q&A

Deep Learning qua các con số (Ước tính)

👥 Nhà phát triển	2.5+ Triệu
📁 Framework phổ biến	TensorFlow, PyTorch, Keras, JAX
📈 Dữ liệu huấn luyện	GBs to TBs
🕒 Thời gian huấn luyện	Minutes to Days
💻 Bộ nhớ GPU	4GB to 80GB+
🎯 Độ chính xác đạt được	90% to 99%+

Chủ đề bao gồm

- ✓ Neural Networks
- ✓ CNN & Computer Vision
- ✓ RNN, LSTM & GRU
- ✓ Transformers & NLP
- ✓ Activation Functions
- ✓ Loss Functions
- ✓ Optimization
- ✓ Regularization
- ✓ Transfer Learning
- ✓ Generative Models
- ✓ Model Evaluation
- ✓ Deployment & MLOps
- ✓ Best Practices

🎓 Tài liệu này dành cho ai?

- ★ Học sinh, sinh viên & người mới bắt đầu
- ★ Kỹ sư & Lập trình viên ML
- ★ Bất kỳ ai đam mê AI

“Cách tốt nhất để dự đoán tương lai là
tự tạo ra nó cùng **Deep Learning**.”



DEEP LEARNING

Bước tiếp theo sau ML Basics trên hành trình AI

TRANG 1

"Deep Learning là một nhánh con của Machine Learning, được lấy cảm hứng từ cấu trúc và chức năng của bộ não con người, gọi là Artificial Neural Networks."

★ 1. DEEP LEARNING LÀ GÌ?

- Deep Learning (DL) là một phần của Machine Learning (ML), sử dụng Artificial Neural Networks (ANNs) với nhiều lớp (hidden layers) để học các pattern từ dữ liệu.
- Nó có thể tự động học các feature từ dữ liệu thô mà không cần feature engineering thủ công.
- Nó hoạt động cực kỳ hiệu quả với lượng dữ liệu lớn và các bài toán phức tạp.

★ 2. TẠI SAO CẦN DEEP LEARNING?

- Xử lý tốt **unstructured data** như hình ảnh, âm thanh, văn bản, video.
- Tự động học các **hierarchical feature**.
- Độ chính xác** cao trong các tác vụ phức tạp.
- Mở rộng tốt cùng **big data** và sức mạnh tính toán.
- Là sức mạnh đứng sau các ứng dụng **AI** hiện đại.

3. ML vs DL

Khía cạnh	Machine Learning (ML)	Deep Learning (DL)
Feature Engineering	Cần feature engineering thủ công	Tự động học feature từ dữ liệu
Yêu cầu dữ liệu	Hoạt động tốt với dataset nhỏ đến vừa	Cần lượng dữ liệu lớn để đạt hiệu suất tốt nhất
Mô hình	Shallow model (vd: Linear Regression, SVM, Decision Tree)	Deep Neural Network với nhiều hidden layer
Hiệu suất	Tốt cho bài toán đơn giản	Xuất sắc cho bài toán phức tạp
Tính toán	Ít tốn tài nguyên tính toán hơn	Cao, cần GPU/TPU
Ví dụ	Phát hiện spam, dự đoán giá nhà	Nhận diện hình ảnh, giọng nói, xe tự lái, ChatGPT,...

DL trong thực tế

-  Nhận diện hình ảnh
-  Nhận diện giọng nói
-  Xe tự lái
-  Chatbot & NLP

★ 4. ỨNG DỤNG THỰC TẾ



Gợi ý YouTube/
Netflix



Gợi ý E-commerce



Trợ lý giọng nói
(Siri, Alexa)



Phát hiện Email
spam



Phân tích hình
ảnh y tế



Chatbot AI
(ChatGPT)

SỨ MỆNH: Ghi chú DSA trường tồn - cách tư duy bằng code trong những giới hạn.

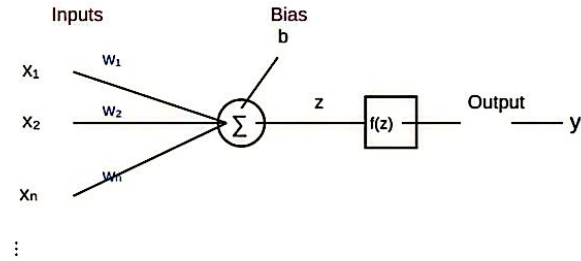
2. NEURAL NETWORK FUNDAMENTALS

TRANG 2

Các mô hình Deep Learning được xây dựng bằng **Artificial Neural Networks (ANNs)**, lấy cảm hứng từ bộ não con người.

★ 2.1 NEURON (PERCEPTRON)

Neuron là đơn vị cơ bản cấu tạo nên một neural network.



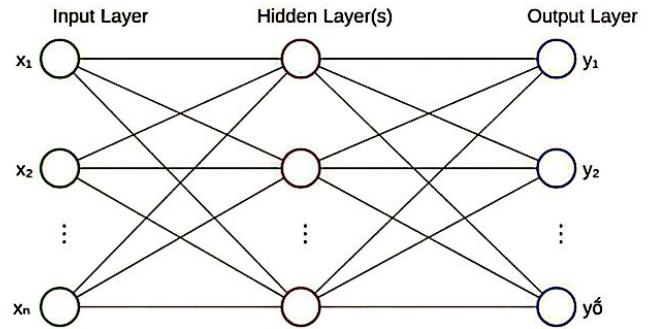
x_i = inputs
 w_i = weights
 b = bias
 z = weighted sum

$f(z)$ = activation function

Mathematically,
 $z = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$
 $y = f(z)$

★ 2.2. CÁC LAYER TRONG NEURAL NETWORK

- **Input Layer:** Nhận dữ liệu thô.
- **Hidden Layer(s):** Thực hiện tính toán và trích xuất feature.
- **Output Layer:** Cho ra kết quả dự đoán cuối cùng.



Một mạng có thể có nhiều hidden layer.
Nhiều layer hơn → học được pattern phức tạp hơn.

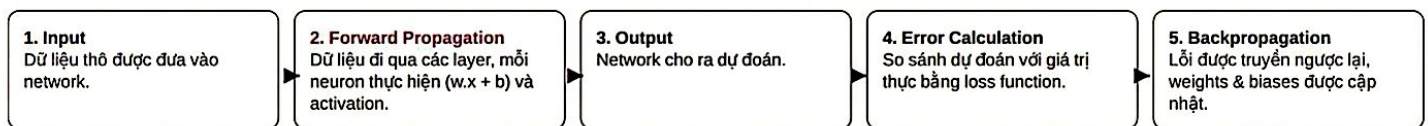
★ 2.3 WEIGHTS & BIASES

- **Weights (w_i)** kiểm soát độ mạnh của kết nối giữa các neuron.
- **Bias (b)** cho phép mô hình dịch chuyển activation function.
- Trong quá trình training, **weights** và **biases** được học để giảm thiểu lỗi (error).

★ 2.4 CÁC LOẠI NEURAL NETWORK (VÍ DỤ)

Feedforward NN	Thông tin chỉ di chuyển về phía trước (Input → Output).
CNN (Convolutional NN)	Dùng cho dữ liệu hình ảnh.
RNN (Recurrent NN)	Dùng cho dữ liệu tuần tự / time series.
LSTM / GRU	Loại RNN đặc biệt, xử lý long-term dependencies.
Transformers	Dùng trong NLP, Computer Vision và nhiều mô hình AI hiện đại.

★ 2.5 NEURAL NETWORK DỰ ĐOÁN NHƯ THẾ NÀO?



Quá trình này lặp lại qua nhiều iteration cho đến khi mô hình học được!

Tip: Hãy hình dung neural network như một **function approximator** - học ánh xạ giữa Input (X) và Output (Y) từ dữ liệu.

Trang tiếp theo →
Activation Functions

3. ACTIVATION FUNCTIONS

TRANG 3

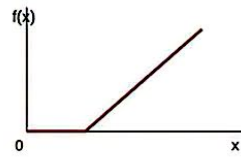
Activation function thêm **non-linearity** vào neural network, giúp nó học được các **pattern phức tạp**.

★ Nếu không có activation function, một neural network chỉ là một mô hình **linear regression**.

① ReLU

(Rectified Linear Unit)

$$f(x) = \max(0, x)$$



Range: $[0, \infty)$

Pros:

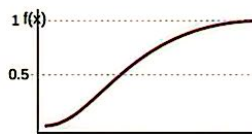
- Đơn giản & nhanh
- Giúp giảm vanishing gradient

Cons:

- Dying ReLU problem (neuron có thể "chết")
 - Không zero-centered
- Dùng trong: Hidden layers

② Sigmoid

$$f(x) = 1 / (1 + e^{-x})$$



Range: $(0, 1)$

Pros:

- Output từ 0 đến 1
- Hữu ích cho probability

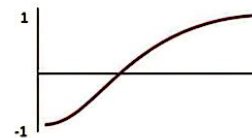
Cons: Vanishing gradient, không zero-centered

Dùng trong: Output layer (binary classification), Gates trong LSTM/GRU

③ Tanh

(Hyperbolic Tangent)

$$f(x) = (e^x - e^{-x}) / (e^x + e^{-x})$$



Range: $(-1, 1)$

Pros:

- Zero-centered
- Mượt hơn sigmoid

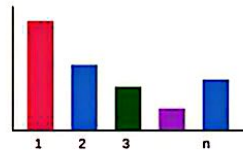
Cons: Vanishing gradient

Dùng trong: Hidden layers, mạng cũ (nay chủ yếu dùng ReLU)

④ Softmax

(Multi-class output)

$$f(z_i) = e^{z_i} / \sum e^{z_k}$$



Range: $(0, 1)$, tổng=1

Pros:

- Chuyển score thành probability
- Dùng cho multi-class classification

Cons: Không dùng trong hidden layers

Dùng trong: Output layer cho bài toán multi-class

★ TÓM TẮT SO SÁNH

Function	Formula	Range	Zero-Centered?	Dùng tốt nhất trong
ReLU	$\max(0, x)$	$[0, \infty)$	Không	Hidden Layers (Mặc định)
Sigmoid	$1/(1+e^{-x})$	$(0, 1)$	Không	Output (Binary), Gates
Tanh	$(e^x - e^{-x}) / (e^x + e^{-x})$	$(-1, 1)$	Có	Hidden Layers
Softmax	$e^{z_i} / \sum e^{z_k}$	$(0, 1)$, tổng=1	Không	Output (Multi-class)

TIPS

- Dùng **ReLU** trong hidden layers để training tốt hơn.
- Dùng **Softmax** cho output multi-class.
- Chọn activation function phù hợp với loại bài toán!

★ Ghi nhớ:

Chọn đúng activation function có thể tạo ra khác biệt lớn trong khả năng học của mô hình.

4. FORWARD & BACKPROPAGATION

TRANG 4

Forward Propagation là cách mô hình đưa ra dự đoán.

Backpropagation là cách mô hình **học** từ những sai lầm của nó.

★ 4.1 FORWARD PROPAGATION

Dữ liệu chảy từ input layer → hidden layer(s) → output layer để tạo ra kết quả cuối cùng.

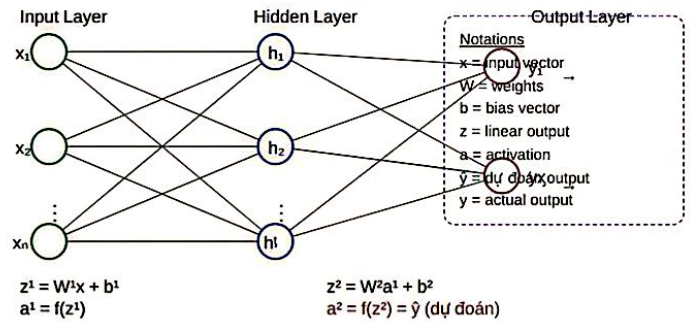
Với một neuron đơn lẻ:

$$z = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$$
$$a = f(z) \quad (a = \text{activation/output})$$

z = weighted sum (phần linear)

a = activation (sau khi áp dụng activation function)

$f()$ = activation function



★ 4.2 BACKPROPAGATION (TỔNG QUAN)

Backpropagation hoạt động **ngược lại** từ output đến input layer để cập nhật weights và biases nhằm giảm thiểu error.

1. Tính loss bằng predicted và actual output.

2. Tính error (gradient) tại output layer.

3. Lan truyền error ngược qua các hidden layer.

4. Cập nhật weights & biases bằng gradient.

Tại sao nó hoạt động?
Calculus (Chain Rule) giúp tính từng weight đóng góp thế nào vào lỗi cuối cùng.

Mục tiêu: Giảm thiểu loss function bằng cách điều chỉnh weights và biases.

★ 4.3 BACKPROPAGATION (CHI TIẾT)

1. Output Layer Error

• Cho regression (MSE):

$$\delta^{[L]} = (\hat{y} - y) \odot f'(z^{[L]})$$

• Cho classification (Cross Entropy với Softmax):

$$\delta^{[L]} = \hat{y} - y$$

$\odot$ = element-wise multiplication

2. Hidden Layer Error

$$\delta^{[l]} = (W^{[l+1]})^T \delta^{[l+1]} \odot f'(z^{[l]})$$

Trong đó,
 l = layer hiện tại
 L = layer cuối cùng

3. Gradient (để cập nhật)

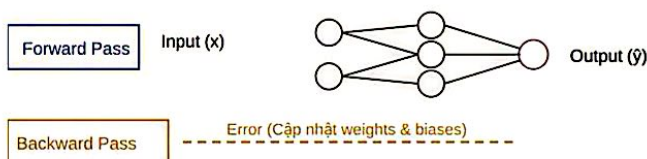
$$dW^{[l]} = \delta^{[l]} (a^{[l-1]})^T$$
$$db^{[l]} = \delta^{[l]}$$

W, b được cập nhật bằng Gradient Descent

$$W^{[l]} = W^{[l]} - \eta dW^{[l]}$$
$$b^{[l]} = b^{[l]} - \eta db^{[l]}$$

η = learning rate

4.4 TÓM TẮT TRỰC QUAN (Prediction)



TIPS

- ☆ Hiểu luồng: Forward → Loss → Backward → Update
- ☆ Activation function thêm non-linearity.
- ☆ Backpropagation là ứng dụng của chain rule.
- ☆ Thay đổi nhỏ ở weights có thể giảm loss đáng kể.
- ☆ Luyện tập với ví dụ nhỏ để xây dựng trực giác!

Ghi nhớ: **Forward Propagation** tạo ra dự đoán, **Backpropagation** làm mô hình tốt hơn. ❤️

5. LOSS FUNCTIONS

TRANG 5

Loss function đo lường mức độ sai của dự đoán. Mục tiêu của ta là giảm thiểu loss.

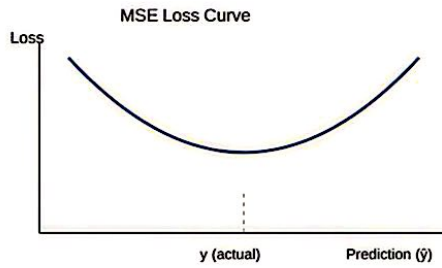
★ Loss thấp hơn → dự đoán tốt hơn → mô hình tốt hơn.

★ 5.1 MEAN SQUARED ERROR (MSE)

Dùng cho bài toán **Regression**.

$$MSE = (1/n) \sum (y_i - \hat{y}_i)^2$$

y_i = giá trị thực
 $\hat{y}_i$ = giá trị dự đoán
 n = số lượng mẫu



Pros

- Phạt các lỗi lớn nhiều hơn.
- Mượt và khả vi (differentiable).

Cons

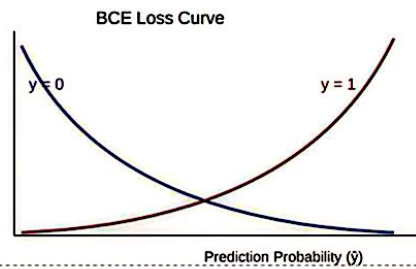
- Nhạy cảm với outlier.
- Không lý tưởng cho classification.

★ 5.2 BINARY CROSS ENTROPY (LOG LOSS)

Dùng cho **Binary Classification** (0 hoặc 1).

$$BCE = -(1/n) \sum [y_i \log(\hat{y}_i) + (1-y_i) \log(1-\hat{y}_i)]$$

$y_i \in \{0,1\}$ (nhãn thực)
 $\hat{y}_i \in (0,1)$ (probability dự đoán)



Pros

- Tốt nhất cho binary classification.
- Xử lý probability tốt.

Cons

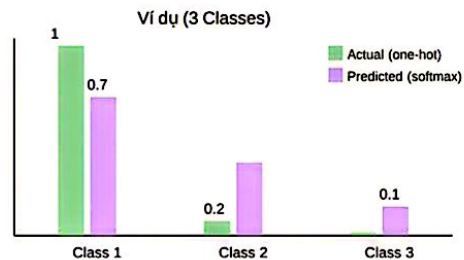
- Không dùng cho regression.
- Chỉ cho output binary.

★ 5.3 CATEGORICAL CROSS ENTROPY

Dùng cho **Multi-class Classification** (2 lớp trở lên).

$$CCE = -(1/n) \sum y_{ij} \log(\hat{y}_{ij})$$

C = số lượng class
 $y_{ij} = 1$ nếu mẫu i thuộc class j , ngược lại $= 0$
 $\hat{y}_{ij}$ = probability dự đoán class j cho mẫu i



Pros

- Hoàn hảo cho multi-class classification.
- Hoạt động tốt với softmax output.

Cons

- Không dùng cho binary (dùng BCE).
- Nhạy cảm với label nhiễu.

SO SÁNH NHANH

Loss Function	Dùng cho	Loại Output	Range	Dùng tốt nhất khi
MSE	Regression	Continuous	$[0, \infty)$	Dự đoán số (giá, chiều cao,...)
Binary Cross Entropy	Binary Classification	Probability (0/1)	$[0, \infty)$	Yes/No, Spam/Not Spam
Categorical Cross Entropy	Multi-class Classification	Probability (multi-class)	$[0, \infty)$	Cat/Dog/Bird classification

💡 Quy tắc ngón tay cái: Regression → MSE | Binary Classification → BCE | Multi-class Classification → CCE

6. GRADIENT DESCENT & OPTIMIZATION

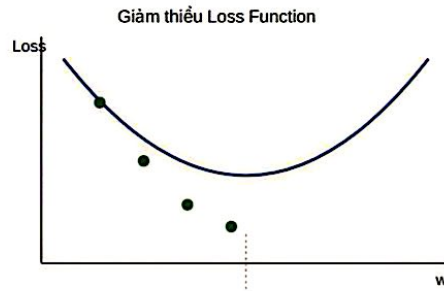
TRANG 6

Gradient Descent là một thuật toán **optimization** dùng để **giảm thiểu** loss function bằng cách cập nhật các tham số theo hướng ngược với gradient.

★ Mục tiêu: Tìm tập tham số (weights, biases) giúp giảm thiểu loss.

★ 6.1 GRADIENT DESCENT LÀ GÌ?

- Bắt đầu với random weights.
- Tính **gradient** (độ dốc) của loss function.
- Cập nhật tham số theo hướng **ngược lại** với gradient.
- Lặp lại cho đến khi loss được giảm thiểu.



Quy tắc cập nhật

$$w_{t+1} = w_t - \eta \nabla L(w_t)$$

w_t = weights hiện tại

η = learning rate

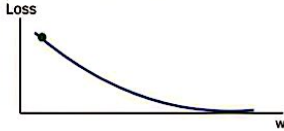
$\nabla L(w_t)$ = gradient của loss function theo weights

★ 6.2 CÁC LOẠI GRADIENT DESCENT

1. Batch Gradient Descent

Dùng toàn bộ dataset để tính gradient.

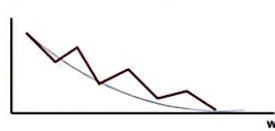
Cập nhật 1 lần mỗi epoch.
• Ổn định nhưng chậm hơn.



2. Stochastic Gradient Descent

Dùng một training example mỗi lần.

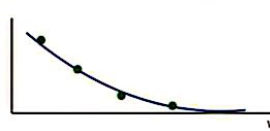
Cập nhật cho mỗi example.
• Nhanh hơn nhưng nhiễu.



3. Mini-Batch Gradient Descent

Dùng một batch nhỏ các example.

Cập nhật cho mỗi mini-batch.
• Cân bằng tốt giữa tốc độ và ổn định.



4. Online Gradient Descent

Tương tự SGD, dùng cho dữ liệu online/streaming.

Cập nhật mô hình liên tục khi dữ liệu đến.
• Thích ứng nhanh.



★ 6.3 CÁC THUẬT TOÁN OPTIMIZATION

1. SGD (Vanilla GD)

Dạng đơn giản nhất. Baseline tốt.

$$w_{t+1} = w_t - \eta \nabla L(w_t)$$

2. Momentum

Tăng tốc theo hướng phù hợp & giảm dao động.

$$v_t = \gamma v_{t-1} + \eta \nabla L(w_t)$$
$$w_{t+1} = w_t - v_t$$

3. RMSProp

Điều chỉnh learning rate cho từng tham số. Tốt cho RNN.

$$s_t = \beta s_{t-1} + (1 - \beta)(\nabla L)^2$$

4. Adam

Kết hợp Momentum & RMSProp. Optimizer phổ biến nhất.

m_t, v_t (bias correction)

TIPS

- Bắt đầu với Adam (mặc định).
- Điều chỉnh cẩn thận learning rate (η).
- Quá cao $\rightarrow$ vọt qua minima.
- Quá thấp $\rightarrow$ hội tụ chậm.

★ 6.4 SO SÁNH CÁC OPTIMIZER

Optimizer	Memory	Tốc độ	Hội tụ	Dùng tốt nhất trong	Ghi chú
SGD	Thấp	Nhanh (mỗi update)	Chậm	Dataset lớn	Đơn giản, có thể dao động
Momentum	Trung bình	Nhanh hơn	Tốt hơn	CNN, Deep Nets	Giảm dao động
RMSProp	Trung bình	Nhanh	Tốt	RNN, bài toán non-stationary	Xử lý gradient thay đổi
Adam	Cao	Nhanh	Xuất sắc	Hầu hết bài toán Deep Learning	Lựa chọn mặc định tốt nhất

★ Ghi nhớ: Optimization là chìa khoá để training mô hình hiệu quả. ❤️

7. OVERFITTING & REGULARIZATION

TRANG 7

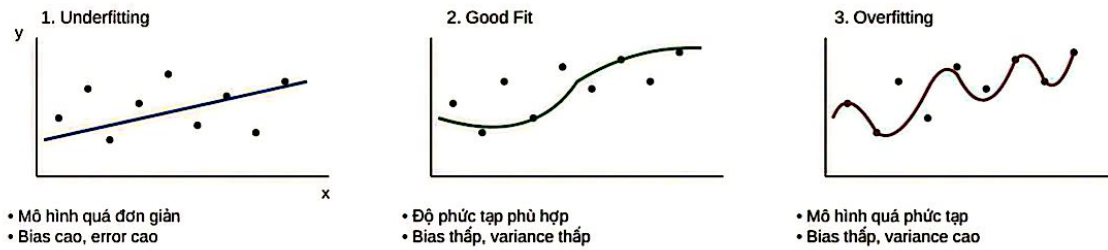
Overfitting xảy ra khi mô hình học **quá kỹ** training data, bao gồm cả noise, và hoạt động kém trên **dữ liệu** chưa từng thấy.

Tại sao xảy ra?

- Mô hình quá phức tạp
- Quá nhiều tham số
- Quá ít training data
- Training quá lâu

★ Mục tiêu: Xây dựng mô hình tổng quát hoá tốt trên dữ liệu mới.

★ 7.1 HIỂU VỀ OVERFITTING



Đánh đổi Bias vs Variance

Tăng độ phức tạp

Bias giảm

Variance tăng

Cần sự cân bằng phù hợp!

★ 7.2 CÁC KỸ THUẬT REGULARIZATION

1. L2 Regularization (Ridge)

Thêm penalty bằng bình phương độ lớn của weights.

$$L = L_0 + \lambda \sum w_i^2$$

- Thu nhỏ weight lớn
- Giảm overfitting
- λ = độ mạnh regularization

2. L1 Regularization (Lasso)

Thêm penalty bằng giá trị tuyệt đối của weights.

$$L = L_0 + \lambda \sum |w_i|$$

- Có thể đưa weight về 0
- Tạo mô hình sparse
- Tốt cho feature selection

3. Dropout

Ngẫu nhiên loại bỏ (set = 0) một số neuron khi training.

- Ngăn co-adaptation
- Hoạt động như training nhiều mạng con

4. Early Stopping

Dừng training khi hiệu suất trên validation set ngừng cải thiện.

- Đơn giản nhưng hiệu quả
- Ngăn overfitting

5. Batch Normalization

Chuẩn hoá output của một layer về mean 0, variance 1.

$$\hat{x} = (x - \mu) / \sqrt{\sigma^2 + \epsilon}$$

- Ổn định & tăng tốc training
- Có vai trò như regularizer

6. Data Augmentation

Tăng nhân tạo dữ liệu training bằng các phép biến đổi.

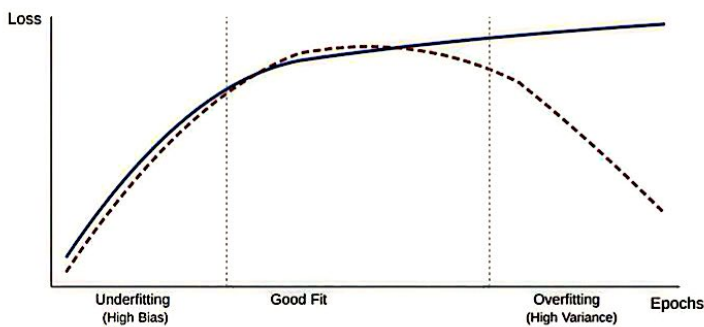
- Cải thiện khả năng tổng quát hoá
- Phổ biến với ảnh (flip, crop, rotate, zoom, noise,...)

7. Đơn giản hoá Model

Dùng mô hình đơn giản hơn hoặc giảm số tham số.

- Ít layer hơn
- Ít neuron hơn
- Bỏ feature không cần thiết

★ 7.3 TRAINING CURVE (PHÁT HIỆN)



Quan sát	Training Loss	Validation Loss	Kết luận
Cả hai đều cao	Cao	Cao	Underfitting (Tăng model capacity)
Cả hai đều thấp	Thấp	Thấp	Good Fit (Mô hình tổng quát hoá tốt)
Train thấp, Val cao	Thấp	Cao	Overfitting (Áp dụng Regularization)

💡 **Điểm mấu chốt:** Đừng chỉ fit dữ liệu, hãy hiểu nó. Regularization giúp mô hình tổng quát hoá và hoạt động tốt trên dữ liệu chưa từng thấy. ❤️

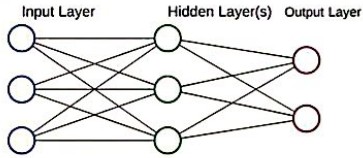
8. NEURAL NETWORK ARCHITECTURES

TRANG 8

Mỗi architecture được thiết kế để xử lý hiệu quả các loại dữ liệu và tác vụ học khác nhau.

1. FEEDFORWARD NEURAL NETWORK (FNN)

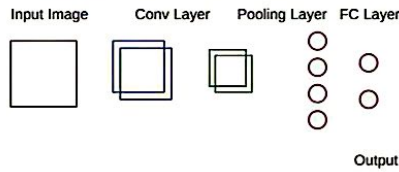
- Dữ liệu chỉ đi về phía trước, từ input layer đến output layer.
- Không có cycle hay loop.



Dùng cho: Classification, Regression, Tabular data

2. CONVOLUTIONAL NEURAL NETWORK (CNN)

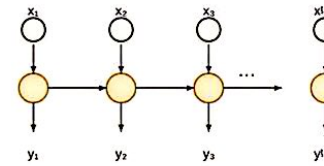
- Dùng convolutional layer để trích xuất spatial feature.
- Tốt nhất cho dữ liệu hình ảnh và video.



Dùng cho: Nhận diện hình ảnh, Object detection, Medical imaging

3. RECURRENT NEURAL NETWORK (RNN)

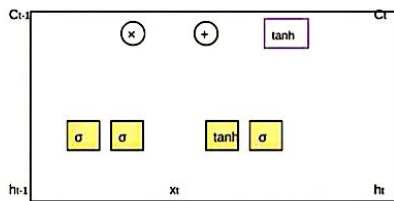
- Thiết kế cho dữ liệu tuần tự.
- Có loop để duy trì hidden state (bộ nhớ).



Dùng cho: Text, Speech, Time series, Language modeling

4. LSTM (LONG SHORT-TERM MEMORY)

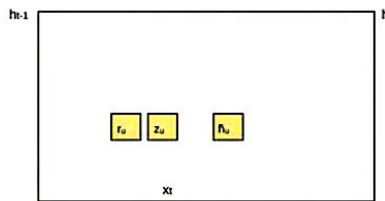
- Khắc phục vấn đề vanishing gradient của RNN.
- Dùng gate để kiểm soát luồng thông tin.



Dùng cho: Chuỗi dài, Translation, Video analysis

5. GRU (GATED RECURRENT UNIT)

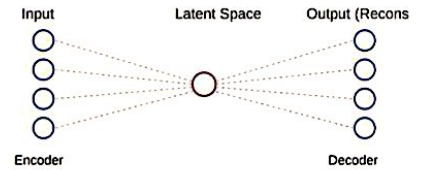
- Phiên bản đơn giản hoá của LSTM.
- Có reset gate và update gate.



Dùng cho: Tương tự LSTM nhưng nhanh hơn và ít phức tạp hơn

6. AUTOENCODER

- Mô hình unsupervised learning.
- Học nén dữ liệu rồi tái tạo lại.

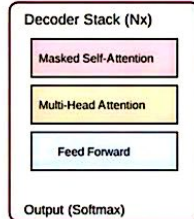
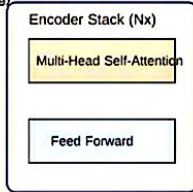


Dùng cho: Dimensionality reduction, Denoising, Anomaly detection

7. TRANSFORMER (ATTENTION BASED MODEL)

- Dùng self-attention mechanism thay vì recurrence.
- Highly parallelizable, hoạt động tốt trên chuỗi dài.

Input (Embedding)



Dùng cho: NLP (BERT, GPT), Translation, Summarization

SO SÁNH CÁC ARCHITECTURE

Architecture	Tốt nhất cho	Xử lý Sequence?	Nắm bắt Spatial Info?	Tốc độ Training
FNN	Tabular Data	Không	Không	Nhanh
CNN	Images/Videos	Không	Có	Trung bình
RNN	Sequences	Có	Không	Chậm
LSTM	Long Sequences	Có	Không	Chậm
GRU	Sequences	Có	Không	Nhanh hơn
Autoencoder	Unsupervised Learning	Không	Không	Trung bình
Transformer	NLP/Long Sequences	Có	Không	Nhanh (Parallel)

💡 **Điểm mấu chốt:** Lựa chọn architecture phụ thuộc vào loại dữ liệu và bài toán bạn đang giải quyết.
Hiểu dữ liệu → Chọn đúng architecture → Training hiệu quả → Đạt kết quả tốt hơn. ☆

9. CONVOLUTIONAL NEURAL NETWORKS (CNN)

TRANG 9

CNN được thiết kế để tự động học **spatial hierarchy** của các feature từ hình ảnh bằng convolution operation.

Tại sao dùng CNN?

- Tận dụng spatial structure của dữ liệu
- Ít tham số hơn
- Translation invariance
- Rất tốt cho hình ảnh & video

★ 9.1 CONVOLUTION OPERATION

Một filter (kernel) nhỏ trượt qua input và tính element-wise multiplication rồi cộng lại.

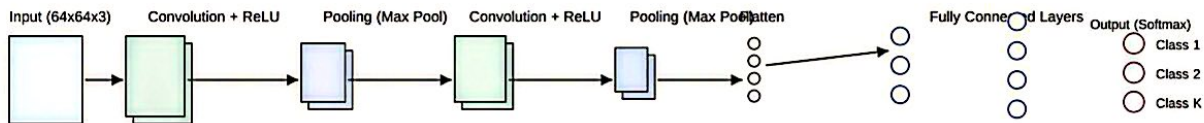
Công thức Output size:

$$O = (N - F + 2P) / S + 1$$

N=input size, F=filter size
P=padding, S=stride

Input Image (5x5)		Filter/Kernel (3x3)		Feature Map (Output 3x3)																																											
<table><tr><td>1</td><td>1</td><td>1</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td><td>1</td><td>0</td></tr><tr><td>0</td><td>0</td><td>1</td><td>1</td><td>1</td></tr><tr><td>0</td><td>0</td><td>1</td><td>1</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td><td>0</td><td>0</td></tr></table>	1	1	1	0	0	0	1	1	1	0	0	0	1	1	1	0	0	1	1	0	0	1	1	0	0	*	<table><tr><td>1</td><td>0</td><td>-1</td></tr><tr><td>1</td><td>0</td><td>-1</td></tr><tr><td>1</td><td>0</td><td>-1</td></tr></table>	1	0	-1	1	0	-1	1	0	-1	=	<table><tr><td>0</td><td>-1</td><td>-1</td></tr><tr><td>3</td><td>0</td><td>-3</td></tr><tr><td>3</td><td>1</td><td>-1</td></tr></table>	0	-1	-1	3	0	-3	3	1	-1
1	1	1	0	0																																											
0	1	1	1	0																																											
0	0	1	1	1																																											
0	0	1	1	0																																											
0	1	1	0	0																																											
1	0	-1																																													
1	0	-1																																													
1	0	-1																																													
0	-1	-1																																													
3	0	-3																																													
3	1	-1																																													

★ 9.2 CNN ARCHITECTURE (LUỒNG ĐIỂN HÌNH)



Conv: Trích xuất feature → Pool: Giảm spatial size → FC: Học pattern bậc cao → Softmax: Dự đoán class

★ 9.3 CÁC LAYER PHỔ BIẾN TRONG CNN

- Convolution Layer:** Áp dụng filter để trích xuất feature.
- Activation Layer (ReLU):** Thêm non-linearity.
- Pooling Layer (Max/Average):** Giảm spatial size.
- Batch Normalization:** Chuẩn hoá activation, tăng tốc training.
- Dropout:** Ngăn co-adaptation.
- Fully Connected Layer:** Thực hiện classification cuối cùng.
- Softmax Layer:** Chuyển score thành probability.

★ 9.4 VÍ DỤ POOLING (MAX POOLING 2x2, STRIDE=2)

Input Feature Map (4x4)		Output (2x2)																				
<table><tr><td>1</td><td>3</td><td>2</td><td>0</td></tr><tr><td>4</td><td>6</td><td>6</td><td>1</td></tr><tr><td>0</td><td>2</td><td>1</td><td>2</td></tr><tr><td>3</td><td>1</td><td>0</td><td>5</td></tr></table>	1	3	2	0	4	6	6	1	0	2	1	2	3	1	0	5	Max Pool 2x2, s=2	<table><tr><td>6</td><td>2</td></tr><tr><td>3</td><td>5</td></tr></table>	6	2	3	5
1	3	2	0																			
4	6	6	1																			
0	2	1	2																			
3	1	0	5																			
6	2																					
3	5																					

★ 9.5 CÁC KIẾN TRÚC CNN PHỔ BIẾN

Architecture	Năm	Ý tưởng chính	Tốt nhất cho
LeNet-5	1998	CNN đầu tiên cho nhận diện chữ số	Chữ số viết tay
AlexNet	2012	Deep CNN, ReLU, Dropout	ImageNet classification
VGGNet	2014	Rất sâu với filter 3x3	Tác vụ cần độ chính xác cao
GoogLeNet (Inception)	2014	Inception modules	Trích xuất feature hiệu quả
ResNet	2015	Residual (skip) connections	Mạng rất sâu
EfficientNet	2019	Scale depth, width, resolution	Độ chính xác cao, ít compute

ĐIỂM MẪU CHỐT

- ☆ CNN tận dụng spatial structure của hình ảnh.
- ☆ Convolution trích xuất pattern cục bộ.
- ☆ Pooling giảm size và tính toán.
- ☆ Mạng sâu hơn học được feature phức tạp hơn.
- ☆ Dùng ReLU, BatchNorm, Dropout để training tốt hơn.
- ☆ Softmax cho ra class probability cuối cùng.

📌 Ghi nhớ: CNN biến đổi pixel thô thành feature có ý nghĩa, giúp mô hình tập trung vào "CÁI GÌ" quan trọng, chứ không phải "Ở ĐÂU".

RNN được thiết kế để xử lý dữ liệu tuần tự bằng cách duy trì hidden state (bộ nhớ) lưu lại thông tin từ các bước thời gian trước đó.

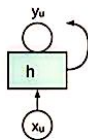
Tại sao dùng RNN?

- Xử lý chuỗi độ dài bất kỳ
- Nắm bắt temporal dependencies
- Chia sẻ tham số theo thời gian
- Lý tưởng cho text, speech & time series

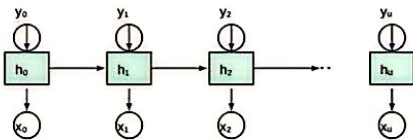
★ Mục tiêu: Xử lý chuỗi độ dài bất kỳ bằng cách ghi nhớ những gì đã xảy ra trước đó.

★ 10.1 RECURRENT NEURAL NETWORK (DẠNG UNFOLD)

Compact Form



Unfolded Form

**10.2 CÔNG THỨC RNN**

$$h_u = f(W_{hx}x_u + W_{hh}h_{u-1} + b_h)$$

$$y_u = g(W_{yh}h_u + b_y)$$

h_u = hidden state tại thời điểm t

x_u = input tại thời điểm t

y_u = output tại thời điểm t

W_{hx}, W_{hh}, W_{yh} = ma trận weight

b_h, b_y = vector bias

f, g = activation function (tanh, ReLU, softmax)

★ 10.3 CÁC LOẠI OUTPUT CỦA RNN

1. One-to-One
(Image Classification)

Single input
Single output

2. One-to-Many
(Image Captioning)

Single input
Multiple outputs

3. Many-to-One
(Sentiment Analysis)

Multiple inputs
Single output

★ 10.4 OUTPUT MANY-TO-MANY

a) Many-to-Many (Đồng bộ)
(POS Tagging)

Input và output sequence có cùng độ dài

b) Many-to-Many (Encoder-Decoder)
(Machine Translation)

Input và output sequence có thể khác độ dài

★ 10.5 ACTIVATION FUNCTION TRONG RNN

tanh

Range: $(-1, 1)$

Thường dùng nhất trong RNN

ReLU

Range: $[0, \infty)$

Training nhanh hơn nhưng có thể kém ổn định

sigmoid

Range: $(0, 1)$

Hữu ích cho gate và output

★ 10.6 VANISHING & EXPLODING GRADIENTS

Vanishing Gradients

Gradient trở nên quá nhỏ khi lan truyền ngược.
(Mô hình quên thông tin dài hạn)

Exploding Gradients

Gradient trở nên quá lớn khi lan truyền ngược.
(Mô hình trở nên không ổn định)

Giải pháp: Dùng LSTM/GRU, Gradient Clipping, Khởi tạo hợp lý, Layer Normalization

★ 10.7 SO SÁNH CÁC BIẾN THỂ RNN

Model	Ý tưởng chính	Long-term Dependencies?	Tốt nhất trong
RNN	Kết nối đệ quy cơ bản	Không (Vanishing)	Chuỗi ngắn
LSTM	Dùng gate (input, forget, output)	Có	Chuỗi dài, NLP, Time series
GRU	LSTM đơn giản hoá (update, reset)	Có	Ứng dụng real-time, Chatbot
BiRNN	Xử lý chuỗi theo cả 2 chiều	Có	Speech, NER, Text understanding

TÓM TẮT

- ✓ RNN xử lý dữ liệu tuần tự bằng hidden state.
- ✓ Chúng gặp vấn đề vanishing/exploding gradient.
- ✓ LSTM và GRU giải quyết vấn đề long-term dependency.
- ✓ Chọn architecture phù hợp với dữ liệu & bài toán.
- ✓ Architecture tốt hơn → Hiệu suất tốt hơn.

💡 **Điểm mấu chốt:** RNN ghi nhớ quá khứ để dự đoán tương lai. Hiểu về sequence chính là chìa khoá để giải quyết các bài toán thực tế. ☆

11. TRANSFORMER (ATTENTION IS ALL YOU NEED)

TRANG 11

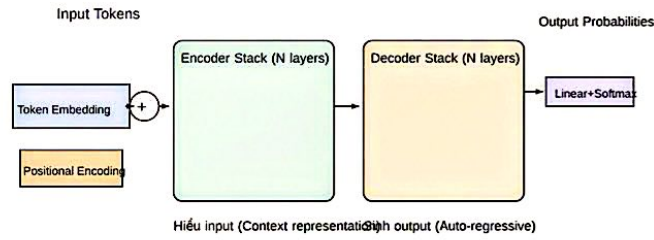
Transformer dùng cơ chế **self-attention** để hiểu mối quan hệ giữa tất cả các từ trong một chuỗi cùng lúc.

★ Mục tiêu: Nắm bắt long-range dependencies hiệu quả và cho phép training song song.

Tại sao dùng Transformer?

- Nắm bắt global context
- Highly parallelizable
- Tốt hơn cho chuỗi dài
- State-of-the-art trong NLP & hơn thế nữa

11.1 BỨC TRANH TỔNG THỂ



11.2 CƠ CHẾ SELF-ATTENTION

Mỗi từ nhìn vào **mọi** từ khác và quyết định mức độ chú ý (attention) cần dành.

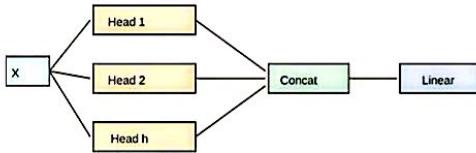


$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / d_k) V$$

Q = Query K = Key V = Value d_k = số chiều của key

11.3 MULTI-HEAD ATTENTION

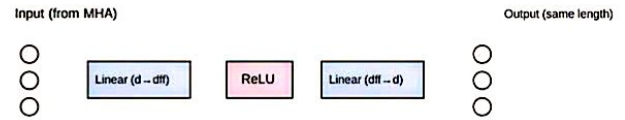
Thay vì dùng một attention, ta dùng nhiều head để tập trung vào các loại quan hệ khác nhau.



$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O$$
$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

11.4 POSITION-WISE FEED FORWARD NETWORK

Sau attention, một fully connected network đơn giản được áp dụng cho mỗi vị trí một cách độc lập.



Thêm non-linearity và giúp học pattern phức tạp.

11.5 ENCODER LAYER (LẶP LẠI N LẦN)

Input → Multi-Head Self-Attention → Add & Norm → Feed Forward Network → Add & Norm → Output

Add & Norm = Residual Connection + Layer Normalization

11.6 DECODER LAYER (LẶP LẠI N LẦN)

Input → Masked Multi-Head Self-Attention → Add & Norm → Multi-Head Cross-Attention (với Encoder Output) → Add & Norm → Feed Forward → Add & Norm

Masked Self-Attention ngăn không cho nhìn thấy token tương lai. Cross-Attention giúp decoder tập trung vào encoder output.

11.7 POSITIONAL ENCODING

Vì transformer không có recurrence, ta thêm thông tin vị trí (positional information). Hai cách phổ biến:

1. Sinusoidal Encoding (Gốc)

$$PE_{(pos, 2i)} = \sin(pos/10000^{2i/d})$$
$$PE_{(pos, 2i+1)} = \cos(pos/10000^{2i/d})$$

2. Learned Embedding (Trainable)

Một vector học được (learnable) được cộng vào token embedding.

Pos	PE ₀	PE ₁	PE ₂	PE ₃
0	0.00	1.00	0.00	1.00
1	0.84	0.54	0.01	1.00
2	0.91	-0.42	0.02	1.00

Positional Encoding được cộng vào Token Embedding

11.8 TRANSFORMER vs RNN

Khía cạnh	RNN	Transformer
Dependency	Tuần tự	Toàn cục (đồng thời)
Training	Khó song song hoá	Song song hoá cao
Chuỗi dài	Gặp khó khăn	Xử lý tốt với attention

ĐIỂM MẪU CHỐT

- Transformer dùng self-attention để nắm bắt global context.
- Multi-head attention cho phép mô hình tập trung vào các khía cạnh khác nhau.

12. TRANSFORMER DECODED

TRANG 12

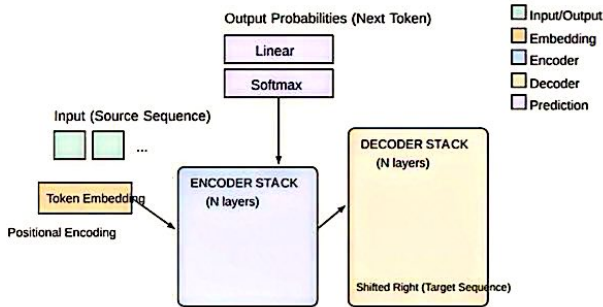
Transformer được xây dựng từ **attention**, nhận biết vị trí (position awareness) và feed forward layer để hiểu và sinh ra chuỗi.

Tại sao dùng Transformer?

- Nắm bắt long-range dependencies
- Xử lý song song (training nhanh hơn)
- Hiệu quả hơn RNN
- Backbone của các mô hình NLP hiện đại

★ Mục tiêu: Xây dựng mô hình hiểu context và mối quan hệ ở quy mô lớn.

12.1 TỔNG QUAN KIẾN TRÚC TRANSFORMER



12.2 SCALED DOT-PRODUCT ATTENTION

Cho Query (Q), Key (K), Value (V):

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / d_k) V$$

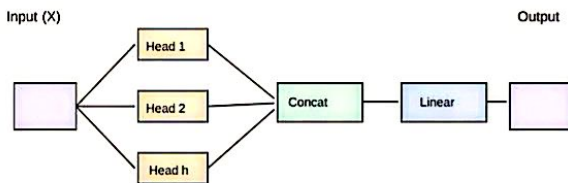
Q = ma trận Query K = ma trận Key

V = ma trận Value d_k = số chiều của key

Tại sao scale bởi $\sqrt{d_k}$?

Để tránh dot product lớn đẩy softmax vào vùng gradient cực nhỏ.

12.3 MULTI-HEAD ATTENTION



$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O$$
$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

12.4 POSITIONAL ENCODING

Vì transformer không có recurrence, positional encoding thêm thông tin về vị trí token.

Hai loại phổ biến:

1. Sinusoidal (Gốc)

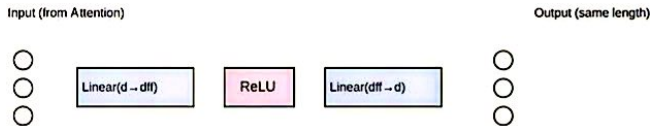
$$PE_{(pos, 2i)} = \sin(pos/10000^{2i/d})$$
$$PE_{(pos, 2i+1)} = \cos(pos/10000^{2i/d})$$

2. Learned (Trainable)

Một vector học được cộng vào token embedding.

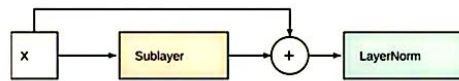
Pos	PE ₁	PE ₂	PE ₃
0	0.00	1.00	0.02
1	0.84	0.54	0.99
2	0.91	-0.42	-0.07

12.5 FEED FORWARD NETWORK (FFN)



Áp dụng theo từng vị trí (position-wise) và giống nhau cho mọi vị trí.
Kích thước tiêu biểu: $d_{model} = 512/768/1024$
 $d_{ff} = 2048/3072/4096$ ($\approx 4 \times d_{model}$)

12.6 LAYER NORMALIZATION & RESIDUAL



$$\text{Output} = \text{LayerNorm}(X + \text{Sublayer}(X))$$

- Residual connection giúp gradient lan truyền tốt hơn.
- LayerNorm ổn định training và tăng tốc hội tụ.

12.7 ENCODER vs DECODER LAYER

ENCODER LAYER: Input → Multi-Head Self-Attention → Add & Norm → Feed Forward → Add & Norm

DECODER LAYER: Input → Masked Multi-Head Self-Attention → Add & Norm → Multi-Head Cross-Attention (Encoder Output) → Add & Norm → Feed Forward → Add & Norm

Encoder dùng self-attention để hiểu context bên trong input.

Decoder có 2 khối attention: 1. Masked self-attention (ngăn nhìn trước) 2. Cross-attention (tập trung vào encoder output)

12.8 ĐỘ PHỨC TẠP MÔ HÌNH (MỖI LAYER)

Component	Time Complexity	Space Complexity
Self-Attention	$O(n^2d)$	$O(n^2)$
Cross-Attention	$O(nmd)$	$O(nm)$
Feed Forward	$O(nd \cdot d_{ff})$	$O(n \cdot d_{ff})$

n =độ dài chuỗi, d =model dim, d_{ff} =feed forward dim, m =độ dài encoder output

12.9 ỨNG DỤNG PHỔ BIẾN

Language Translation Text Summarization Search & QA Systems Speech Recognition Image Captioning

13. TRANSFORMER IN DEPTH

TRANG 13

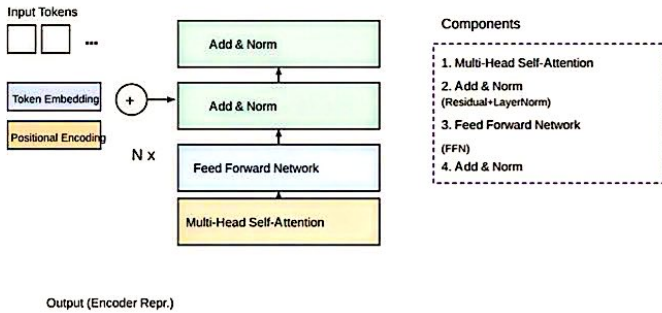
Transformer là **backbone** của NLP hiện đại. Hãy cùng tìm hiểu sâu hơn!

Tại sao Transformer thống trị?

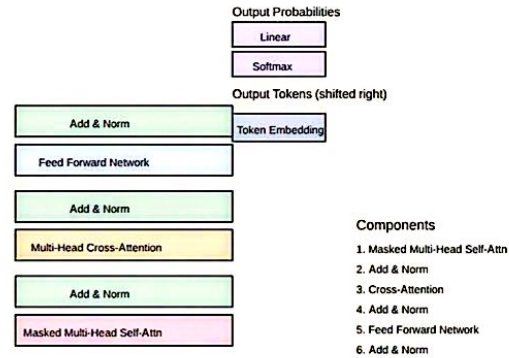
- Nắm bắt long-range dependencies
- Song song hoá (training nhanh hơn)
- Mở rộng cho model & dataset khổng lồ
- State-of-the-art trong NLP, CV, Speech & hơn thế

★ Mục tiêu: Hiểu sâu cách transformer hoạt động, tại sao chúng hiệu quả và được dùng ở đâu.

13.1 KHỐI TRANSFORMER (ENCODER)



13.2 KHỐI TRANSFORMER (DECODER)



13.3 SCALED DOT-PRODUCT ATTENTION

Input: Q (query), K (key), V (value)

$$\text{Attention}(Q,K,V) = \text{softmax}(QK^T/d_k) V$$

Tại sao scale bởi $\sqrt{d_k}$?

- Tránh softmax bị gradient rất nhỏ.
- Giúp training ổn định & hiệu quả.

Ví dụ

$$Q = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \quad K = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix} \quad V = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \quad d_k = 2$$

13.4 MULTI-HEAD ATTENTION



Multi-head attention cho phép mô hình nắm bắt thông tin từ nhiều representation subspace khác nhau tại nhiều vị trí khác nhau.

13.5 POSITIONAL ENCODING

Vì transformer không có recurrence, ta thêm thông tin vị trí. Hai loại:

- Sinusoidal (Cố định)
- Learned (Trainable)

$$PE_{(pos,2i)} = \sin(pos/10000^{2i/d}) \quad PE_{(pos,2i+1)} = \cos(pos/10000^{2i/d})$$

Được cộng vào token embedding.

13.6 MASKED MULTI-HEAD SELF-ATTENTION

Trong decoder, ta che (mask) các token tương lai để ngăn "nhìn trộm"!

	0	1	2	3
0	1	0	0	0
1	1	1	0	0
2	1	1	1	0
3	1	1	1	1

1 = Nhìn thấy được (Visible), 0 = Bị che (Masked)

13.7 LỢI ÍCH CỦA TRANSFORMER

Song song hoá: Training nhanh hơn nhiều so với RNN.

Long-range Dependencies: Attention có thể kết nối các từ ở xa nhau.

Scalability: Mở rộng tốt cho model & dataset lớn.

13.8 ỨNG DỤNG THỰC TẾ

ChatGPT
(Conversational AI)

Google Translate
(Language Translation)

BERT
(Search & QA)

13.9 TÓM TẮT

- ✓ Transformer dùng attention, không dùng recurrence.
- ✓ Self-attention nắm bắt mối quan hệ trong chuỗi.
- ✓ Positional Encoding thêm thông tin thứ tự.
- ✓ Multi-head attention tập trung vào nhiều phần khác nhau.

14. TRAINING TRANSFORMERS

TRANG 14

Training một transformer nghĩa là dạy nó dự đoán next token bằng cách giảm thiểu sai số giữa token dự đoán và thực tế.

- Tại sao Training quan trọng?
- Học pattern từ dữ liệu text khổng lồ
 - Cải thiện hiểu context & semantics
 - Dự đoán next-token chính xác
 - Giúp mô hình tổng quát hoá với dữ liệu mới
 - Backbone cho mọi downstream task

★ Mục tiêu: Giảm thiểu sai số dự đoán để mô hình học pattern, hiểu context và tổng quát hoá với dữ liệu mới.

14.1 MỤC TIÊU TRAINING

Transformer được training bằng Next Token Prediction.



Mô hình học bằng cách giảm thiểu độ khác biệt (error).

So sánh với Actual Next Token (mat) → Tính Loss (Cross-Entropy) → Cập nhật Model Parameters (Backpropagation)

14.2 QUY TRÌNH TRAINING

1. Raw Text
Kho văn bản lớn
2. Tokenization
Chuyển text thành token
3. Input Sequences
Tạo chuỗi độ dài cố định
4. Transformer
Xử lý & dự đoán next token
5. Loss & Update
Tính loss, lan truyền & cập nhật weights

Training Loop (Lặp lại nhiều lần)
Forward Pass → Compute Loss → Backpropagation → Update Weights

14.3 LOSS FUNCTION (CROSS-ENTROPY)

Cross-entropy đo lường sự khác biệt giữa phân phối xác suất dự đoán và next token thực tế.

$$\text{Loss} = -\sum y_i \log(\hat{y}_i)$$

Trực giác:
Loss thấp → mô hình tự tin & đúng.
Loss cao → mô hình sai hoặc không chắc chắn.

V=kích thước vocabulary, y_i =xác suất thực (one-hot), $\hat{y}_i$ =xác suất dự đoán

14.4 OPTIMIZATION

Optimizer dùng để cập nhật tham số mô hình và giảm thiểu loss.

Adam	Learning rate thích ứng
AdamW	Adam + Weight Decay
SGD+Momentum	Momentum tăng tốc

Hyperparameter chính

- Learning Rate (1e-4)
- Batch Size (32,64)
- Optimizer (AdamW)
- Weight Decay (0.01)
- Warmup Steps

14.5 KỸ THUẬT REGULARIZATION

Dropout: Ngẫu nhiên loại bỏ neuron khi training.
Weight Decay: Phạt weight lớn để giảm overfitting.
Label Smoothing: Ngăn mô hình quá tự tin.
Early Stopping: Dừng khi validation loss ngừng cải thiện.

14.6 DỮ LIỆU & SCALING

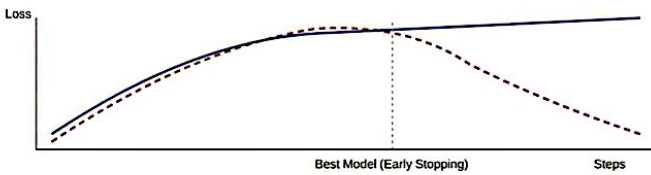
Yêu cầu dữ liệu: Lượng lớn text (GB đến TB), miễn đa dạng, chất lượng cao quan trọng hơn số lượng.
Scaling Laws: Data nhiều hơn/Model lớn hơn/Compute nhiều hơn → Loss thấp hơn (nhưng lợi ích giảm dần).

14.7 TRAINING QUY MÔ LỚN

Distributed Training: Dùng nhiều GPU/TPU.
Data Parallelism: Cùng model, khác data shard.
Model Parallelism: Chia model qua nhiều thiết bị.
Mixed Precision: Dùng FP16/BF16 để training nhanh & tốn ít bộ nhớ hơn.

14.8 ĐÁNH GIÁ TRONG QUÁ TRÌNH TRAINING

Metrics: ✓ Training Loss ✓ Validation Loss ✓ Perplexity (PPL) ✓ Accuracy (cho task cụ thể)



14.9 THÁCH THỨC THƯỜNG GẶP KHI TRAINING

Thách thức	Tại sao xảy ra	Giải pháp
Overfitting	Mô hình học thuộc training data	Thêm data, regularization, early stopping
Underfitting	Model quá nhỏ hoặc training chưa đủ	Model lớn hơn, training nhiều hơn
Vanishing/Exploding Gradients	Khởi tạo sai hoặc model quá sâu	LayerNorm, Residual, khởi tạo tốt
Training chậm	Model lớn, compute hạn chế	Hardware tốt hơn, optimization, mixed precision
Dùng nhiều bộ nhớ	Batch lớn, chuỗi dài	Gradient checkpointing, batch nhỏ hơn, attention hiệu quả

💡 Điểm mấu chốt: Training transformer tốt là sự kết hợp giữa dữ liệu chất lượng, kiến trúc phù hợp và tinh chỉnh optimization cẩn thận. ☆

15. ỨNG DỤNG THỰC TẾ CỦA TRANSFORMERS

TRANG 15

Transformer được dùng ở **khắp mọi nơi**! Hãy cùng xem chúng vận hành các hệ thống AI hiện đại như thế nào.

Tại sao dùng Transformer?

- Xử lý long-range dependencies
- Hiểu context & ý nghĩa
- Hoạt động tốt với dữ liệu quy mô lớn
- Thích ứng với nhiều task & domain
- Không ngừng cải tiến qua nghiên cứu

15.1 NATURAL LANGUAGE PROCESSING (NLP)

Ứng dụng	Mô hình ví dụ	Transformer làm gì
Machine Translation	Google Translate, DeepL, mBERT	Dịch văn bản từ ngôn ngữ này sang ngôn ngữ khác.
Text Summarization	BART, T5, PEGASUS	Tóm tắt bài viết hoặc tài liệu dài.
Question Answering	BERT, RoBERTa, ALBERT	Tìm câu trả lời chính xác từ đoạn văn dài.
Sentiment Analysis	BERT, DistilBERT, XLNet	Hiểu cảm xúc tích cực, tiêu cực hoặc trung lập.
Text Generation	GPT, LLaMA, PaLM	Sinh văn bản giống người, hiểu context.

15.2 COMPUTER VISION (CV)

Ứng dụng	Mô hình ví dụ	Transformer làm gì
Image Classification	ViT, DeiT, Swin Transformer	Phân loại hình ảnh với độ chính xác cao.
Object Detection	DETR, Deformable DETR	Phát hiện và định vị object trong ảnh.
Image Segmentation	SegFormer, Swin-UNet	Hiểu ảnh ở cấp độ pixel.
Video Understanding	Video Transformer, TimeSformer	Hiểu hành động và sự kiện trong video.
Medical Imaging	Swin Transformer, TransUNet	Phân tích X-ray, MRI, CT scan để chẩn đoán.

15.3 SPEECH & AUDIO

Ứng dụng	Mô hình ví dụ	Transformer làm gì
Speech Recognition	Whisper, Wav2Vec 2.0, Conformer	Chuyển giọng nói thành văn bản chính xác.
Text-to-Speech	FastSpeech 2, VITS	Sinh giọng nói tự nhiên từ văn bản.
Speech Translation	SeamlessM4T, SpeechT5	Dịch giọng nói giữa các ngôn ngữ khác nhau.
Speaker Recognition	ECAPA-TDNN + Transformers	Nhận diện và xác minh người nói.

15.4 RECOMMENDATION SYSTEMS

Ứng dụng	Mô hình ví dụ	Transformer làm gì
Product Recommendation	SASRec, BERT4Rec, CoRec	Gợi ý sản phẩm dựa trên hành vi người dùng.
Content Recommendation	YouTube DNN, TransNet	Đề xuất video, bài viết hoặc nội dung.
Personalized Search	BERT, TFRS	Xếp hạng kết quả theo intent người dùng.
News Feed Ranking	Transformer Ranker, DeepRec	Xếp hạng và hiển thị bài viết liên quan nhất.

15.5 ỨNG DỤNG ĐA PHƯƠNG THỨC (MULTI-MODAL)

Ứng dụng	Mô hình ví dụ	Transformer làm gì
Image + Text Understanding	CLIP, VILT, BLIP-2	Hiểu cả hình ảnh và văn bản cùng lúc.
Visual QA (VQA)	LLaVA, VILT, BLIP	Trả lời câu hỏi về hình ảnh.
Document AI	LayoutLM, Donut, DocFormer	Trích xuất thông tin từ tài liệu scan, form, hoá đơn.
Code + Text Understanding	CodeBERT, PLBART, StarCoder	Hiểu và sinh code kèm ngôn ngữ tự nhiên.

15.6 ỨNG DỤNG MỚI NỔI KHÁC

Ứng dụng	Mô hình ví dụ	Transformer làm gì
Scientific Research	SciBERT, Galactica, BioGPT	Phân tích paper, sinh giả thuyết, tóm tắt.
Finance	FinBERT, Longformer, BloombergGPT	Phân tích tin tức thị trường, dự đoán xu hướng.
Robotics	RT-1, PaLM-E, Perceiver IO	Hiểu môi trường và lên kế hoạch hành động.
Healthcare NLP	BioBERT, ClinicalBERT, Med-PaLM	Phân tích hồ sơ y tế, dự đoán kết quả sức khoẻ.

15.7 TẠI SAO TRANSFORMER HOẠT ĐỘNG TỐT? 15.8 THÁCH THỨC & GIỚI HẠN

- ✓ Self-attention nắm bắt mọi mối quan hệ trong dữ liệu.
- ✓ Song song hoá → training nhanh hơn trên phần cứng hiện đại.
- ✓ Mở rộng tốt cho model & dataset khổng lồ.
- ✓ Transfer learning giúp chúng thích ứng với nhiều task.
- ✓ Kết quả state-of-the-art ở hầu hết mọi domain.
- ⚠ Chi phí tính toán cao (training tốn kém).
- ⚠ Cần lượng lớn dữ liệu.
- ⚠ Tốn nhiều bộ nhớ cho chuỗi dài.
- ⚠ Có thể sinh output thiên vị hoặc sai lệch.
- ⚠ Không phải lúc nào cũng dễ diễn giải (black box).

15.9 TƯƠNG LAI

- ★ Mô hình hiệu quả hơn (nhỏ hơn, nhanh hơn, xanh hơn).
- ★ Hiểu long-context tốt hơn.
- ★ Tích hợp multi-modal mạnh mẽ hơn.
- ★ AI cá nhân hoá cho mọi người.
- ★ Transformer + Reasoning = tiềm năng AGI!

15.10 TÓM TẮT

🌟 DÙ Ở ĐÂU CÓ DỮ LIỆU, TRANSFORMER ĐỀU CÓ THỂ GIÚP ÍCH! 🌟



TRANSFORMER TÓM TẮT NHANH

— TÀI LIỆU THAM KHẢO NHANH TOÀN DIỆN NHẤT —

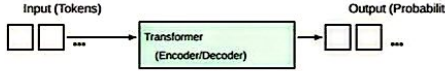
⚡ Ghi nhớ:
Attention is all you need.

1. TRANSFORMER LÀ GÌ?

Transformer là một mô hình deep learning dựa trên cơ chế self-attention để hiểu mối quan hệ giữa tất cả các từ trong một chuỗi cùng lúc.

🎯 Mục tiêu: Nắm bắt long-range dependencies hiệu quả và cho phép training song song.

2. Ý TƯỞNG CỐ LỖI



3. CÔNG THỨC CHÍNH

Scaled Dot-Product Attention

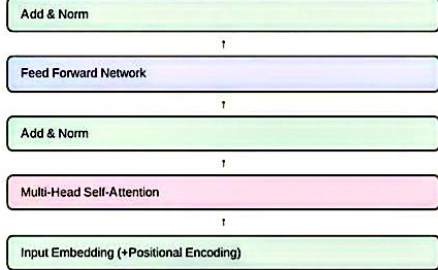
$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k})V$$

Q=ma trận Query K=ma trận Key
V=ma trận Value d_k =số chiều của key

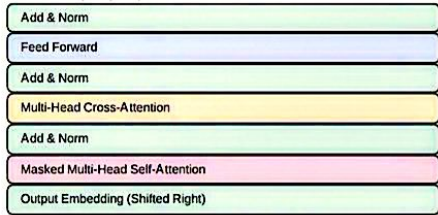
Mục đích: Đo lường mức độ tập trung cần dành từ query đến các key khác nhau.

4. TỔNG QUAN KIẾN TRÚC

ENCODER (N layers)



DECODER (N layers)

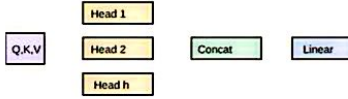


5. CÁC BIẾN THE ATTENTION

a) Scaled Dot-Product Attention

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k})V$$

b) Multi-Head Attention



Multi-Head(Q, K, V) = Concat(head₁, ..., head_h)W^O
head_i = Attention(QW^Q_i, KW^K_i, VW^V_i)

c) Masked Multi-Head Self-Attention (trong Decoder)

Look-ahead mask ngăn attend đến token tương lai.

6. POSITIONAL ENCODING

Vì transformer không có recurrence, positional encoding thêm thông tin vị trí của token.

a) Sinusoidal (Gốc)

$$PE_{(pos, 2i)} = \sin(pos/10000^{2i/d_v})$$
$$PE_{(pos, 2i+1)} = \cos(pos/10000^{2i/d_v})$$

Pos	PE ₀	PE ₁	PE ₂	PE ₃
0	0.00	1.00	0.00	1.00
1	0.84	0.54	0.01	1.00

b) Learned (Trainable)

Một vector học được cộng vào token embedding.

Tại sao dùng Positional Encoding?

✓ Thêm thông tin thứ tự ✓ Giúp hiểu sequence ✓ Tổng quát hoá với chuỗi dài hơn

7. FEED FORWARD NETWORK

Áp dụng theo từng vị trí (position-wise) và giống hệt nhau.



Kích thước: $d_{model} = 512/768/1024$, $d_{ff} = 4 \times d_{model}$

9. ĐỘ PHỨC TẠP (MỖI LAYER)

Component	Time	Space
Self-Attention	$O(n^2d)$	$O(n^2)$
Cross-Attention	$O(nmd)$	$O(nm)$
Feed Forward	$O(nd \cdot d_{ff})$	$O(n \cdot d_{ff})$

n =độ dài chuỗi, d =model dim, d_{ff} =FFN dim, m =độ dài encoder output

10. TRANSFORMER vs RNN

Khía cạnh	RNN	Transformer
Dependency	Tuần tự	Toàn cục
Training	Khó song song	Song song cao
Chuỗi dài	Gặp khó khăn	Xử lý tốt
Tốc độ	Chậm hơn	Nhanh hơn
Bộ nhớ	Cao (trạng thái tuần tự)	Hiệu quả hơn

11. ỨNG DỤNG PHỔ BIẾN

- 🗨️ Language Translation
- 📄 Text Summarization
- ❓ Question Answering
- 🔗 Code Generation
- 🗣️ Speech Recognition
- 🖼️ Image Captioning
- 🔍 Recommendation
- 🔍 Search & Ranking
- 📈 Time Series Forecasting
- 🔬 Scientific Research

12. TRAINING CƠ BẢN

- 📊 Loss Function: Cross-Entropy Loss
- ⚙️ Optimizer: Adam / AdamW
- 📉 Learning Rate: $1e-4$ (warmup + decay)
- 💧 Dropout: 0.1 (phổ biến)
- 📦 Batch Size: 32/64 (tuỳ GPU)
- 🔥 Warmup Steps: Thường 4000-10000
- 🎯 Label Smoothing: 0.1 (giúp tổng quát hoá)
- ⚖️ Weight Decay: 0.01 (phổ biến)

13. THÁCH THỨC & GIỚI HẠN

- ⚠️ Chi phí tính toán cao ($O(n^2)$ trong attention)
- ⚠️ Cần lượng lớn dữ liệu
- ⚠️ Tốn nhiều bộ nhớ cho chuỗi dài
- ⚠️ Có thể sinh output thiên vạy sai lệch
- ⚠️ Không phải lúc nào cũng dễ diễn giải (black box)

14. TẠI SAO TRANSFORMER MẠNH?

- ✓ Self-attention nắm bắt mọi mối quan hệ
- ✓ Hoạt động tốt với dữ liệu quy mô lớn
- ✓ Song song hoá → training nhanh hơn
- ✓ Transfer learning giúp linh hoạt hơn
- ✓ State-of-the-art ở nhiều domain

15. TƯƠNG LAI & XU HƯỚNG

- ★ Mô hình hiệu quả hơn (nhỏ, nhanh, xanh)
- ★ Mô hình hiểu long-context tốt hơn
- ★ Tích hợp multi-modal mạnh hơn
- ★ AI cá nhân hoá
- ★ Transformer + Reasoning = tiềm năng AGI

16. CÔNG THỨC THAM KHẢO NHANH

1. Attention

$$\text{softmax}(QK^T / \sqrt{d_k})V$$

3. Positional Encoding (Sinusoidal)

$$\sin/\cos(pos/10000^{2i/d_v})$$

4. Layer Norm

$$(x - \mu) / \sqrt{\sigma^2 + \epsilon}$$

Notation

$$Q, K, V, d_k, d_{model}, d_{ff}$$